Gradient Descent for Logistic Regression Code

Phần này sẽ viết về ứng dụng của Gradient Descent nhưng cho bài toán Classification/ Logistic Regression

Import các thư viện:

```
import copy, math
import numpy as np
%matplotlib widget
import matplotlib.pyplot as plt
from lab_utils_common import  dlc, plot_data, plt_tumor_data,
sigmoid, compute_cost_logistic
from plt_quad_logistic import plt_quad_logistic, plt_prob
plt.style.use('./deeplearning.mplstyle')
```

Data set trong các numpy array:

```
X_train = np.array([[0.5, 1.5], [1,1], [1.5, 0.5], [3, 0.5],
[2, 2], [1, 2.5]])
y_train = np.array([0, 0, 0, 1, 1, 1])
```

Ta sẽ sử dụng data set này, chốc nữa các điểm với label y = 1 sẽ là chữ x màu đỏ, y = 0 sẽ là chữ o màu xanh nước biển.

```
fig,ax = plt.subplots(1,1,figsize=(4,4))
plot_data(X_train, y_train, ax)

ax.axis([0, 4, 0, 3.5])
ax.set_ylabel('$x_1$', fontsize=12)
```

```
ax.set_xlabel('$x_0$', fontsize=12)
plt.show()
```

- tạo một fig và ax với subplot(1,1) → chỉ có duy nhất 1 hình
- hàm plot_data là được viết sẵn nhưng về bản chất sẽ phác hoạ data của mình thành các điểm, hiển thị legend và tắt header/footer/toolbar cho gọn

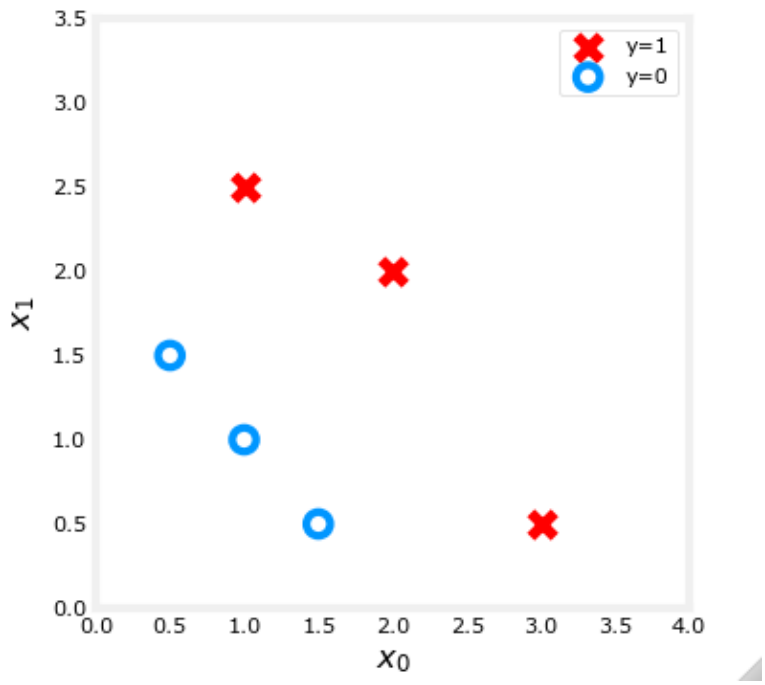
▼ plot_data

```
def plot_data(X, y, ax, pos_label="y=1", neg_label="y=0",
s=80, loc='best' ):
    """ plots logistic data with two axis """
    # Find Indices of Positive and Negative Examples
    pos = y == 1
    neg = y == 0
    pos = pos.reshape(-1,)  #work with 1D or 1D y vectors
    neg = neg.reshape(-1,)

    # Plot examples
    ax.scatter(X[pos, 0], X[pos, 1], marker='x', s=s, c =
'red', label=pos_label)
    ax.scatter(X[neg, 0], X[neg, 1], marker='o', s=s, labe
l=neg_label, facecolors='none', edgecolors=dlblue, lw=3)
    ax.legend(loc=loc)

    ax.figure.canvas.toolbar_visible = False
    ax.figure.canvas.header_visible = False
    ax.figure.canvas.footer_visible = False
```

- .axis là để giới hạn trục x từ 0 tới 4, trục y từ 0 tới 3.5



Logistic Gradient Descent

Nhắc lại lý thuyết về Logistic Gradient Descent (na ná của Linear Regression)

$$
\begin{aligned}
&\text{repeat until convergence: } \{ \\
&\quad w_j = w_j - \alpha \frac{\partial J(\mathbf{w},b)}{\partial w_j} \qquad \text{for j := 0..n-1} \qquad (1) \\
&\quad b = b - \alpha \frac{\partial J(\mathbf{w},b)}{\partial b} \\
&\}
\end{aligned}
$$

Trong đó ta có đạo hàm:

$$\frac{\partial J(\mathbf{w},b)}{\partial w_j} = \frac{1}{m} \sum_{i=0}^{m-1} (f_{\mathbf{w},b}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)} \qquad (2)$$

$$\frac{\partial J(\mathbf{w},b)}{\partial b} = \frac{1}{m} \sum_{i=0}^{m-1} (f_{\mathbf{w},b}(\mathbf{x}^{(i)}) - y^{(i)}) \qquad (3)$$

Lưu ý rằng Model của ta là một Sigmoid Function:

$$f_{\mathbf{w},b}(x) = \frac{1}{1+e^{-z}}$$

Một số lưu ý khi mình tính Gradient(đạo hàm):

- Có nhiều cách để áp dụng thuật toán này nhưng mình sẽ:
  - Khởi tạo 2 biến để tính tổng 2 đạo hàm dj_dw, dj_db
  - Trong đó qua từng example mình sẽ:

  Vòng lặp i: tính độ sai của example đấy bằng Sigmoid Function và tích vô hướng 2 vector:

  $$g(\mathbf{w} \cdot \mathbf{x}^{(i)} + b) - \mathbf{y}^{(i)}$$

  Vòng lặp j: với từng mỗi giá trị input xi_j thì sẽ cộng dj_dw[j] với X[i,j] theo công thức

  - Cộng giá trị sai khác vào giá trị của dj_db (do khi đạo hàm theo bias thì không cần nhân nữa)
  - Sau khi mình chạy vòng lặp xong thì sẽ chia cả dj_db và dj_dw với tổng số example (m)
  - Trả về 2 Gradient

```
def compute_gradient_logistic(X, y, w, b):
    """
    Computes the gradient for linear regression

    Args:
      X (ndarray (m,n): Data, m examples with n features
      y (ndarray (m,)): target values
      w (ndarray (n,)): model parameters
      b (scalar)      : model parameter
    Returns
```

```
      dj_dw (ndarray (n,)): The gradient of the cost w.r.t. t
he parameters w.
      dj_db (scalar)      : The gradient of the cost w.r.t. t
he parameter b.
    """
    m,n = X.shape
    dj_dw = np.zeros((n,))                           #(n,)
    dj_db = 0.

    for i in range(m):
        f_wb_i = sigmoid(np.dot(X[i],w) + b)          #(n,)
(n,)=scalar
        err_i  = f_wb_i  - y[i]                       #scalar
        for j in range(n):
            dj_dw[j] = dj_dw[j] + err_i * X[i,j]      #scalar
        dj_db = dj_db + err_i
    dj_dw = dj_dw/m                                   #(n,)
    dj_db = dj_db/m                                   #scalar

    return dj_db, dj_dw
```

Hàm tính Gradient Descent

- Hàm này tương tự không khác gì với phần giải thích của Linear Regression nhiều biến.

```
def gradient_descent(X, y, w_in, b_in, alpha, num_iters):
    """
    Performs batch gradient descent

    Args:
      X (ndarray (m,n)   : Data, m examples with n features
      y (ndarray (m,))   : target values
      w_in (ndarray (n,)): Initial values of model parameters
      b_in (scalar)      : Initial values of model parameter
      alpha (float)      : Learning rate
```

```
      num_iters (scalar) : number of iterations to run gradie
nt descent

    Returns:
      w (ndarray (n,))   : Updated values of parameters
      b (scalar)         : Updated value of parameter
    """
    J_history = []
    w = copy.deepcopy(w_in)
    b = b_in

    for i in range(num_iters):
        dj_db, dj_dw = compute_gradient_logistic(X, y, w, b)

        w = w - alpha * dj_dw
        b = b - alpha * dj_db

        if i<100000:
            J_history.append( compute_cost_logistic(X, y, w,
b) )

        if i% math.ceil(num_iters / 10) == 0:
            print(f"Iteration {i:4d}: Cost {J_history[-1]}
")

    return w, b, J_history
```

Ta sẽ sử dụng hàm này để tính trên data set của mình:

```
w_tmp  = np.zeros_like(X_train[0])
b_tmp  = 0.
alph = 0.1
iters = 10000

w_out, b_out, _ = gradient_descent(X_train, y_train, w_tmp, b
```

```
_tmp, alph, iters)
print(f"\nupdated parameters: w:{w_out}, b:{b_out}")
```

Output:

Iteration   0: Cost 0.684610468560574

Iteration 1000: Cost 0.1590977666870457

Iteration 2000: Cost 0.08460064176930078

Iteration 3000: Cost 0.05705327279402531

Iteration 4000: Cost 0.04290759421682

Iteration 5000: Cost 0.03433847729884557

Iteration 6000: Cost 0.02860379802212006

Iteration 7000: Cost 0.02450156960879306

Iteration 8000: Cost 0.02142370332569295

Iteration 9000: Cost 0.019030137124109114

updated parameters: w:[5.28 5.08], b:-14.222409982019837

- Dưới đây là phần code để minh hoạ ra kết quả của mình:

Hàm plt_prob được viết sẵn, mặc dù không hiểu rõ lắm nhưng hàm này giúp hình của mình có màu nên theo giá trị xác suất z(0→1) bằng hàm .pcolormesh() Bên phải của hình cũng có một colorbar tương ứng với xác suất bao nhiêu.

▼ plt_prob

```
def plt_prob(ax, w_out,b_out):
    """ plots a decision boundary but include shading to i
ndicate the probability """
    #setup useful ranges and common linspaces
    x0_space  = np.linspace(0, 4 , 100)
    x1_space  = np.linspace(0, 4 , 100)

    # get probability for x0,x1 ranges
    tmp_x0,tmp_x1 = np.meshgrid(x0_space,x1_space)
```

```
    z = np.zeros_like(tmp_x0)
    for i in range(tmp_x0.shape[0]):
        for j in range(tmp_x1.shape[1]):
            z[i,j] = sigmoid(np.dot(w_out, np.array([tmp_x
0[i,j],tmp_x1[i,j]])) + b_out)


    cmap = plt.get_cmap('Blues')
    new_cmap = truncate_colormap(cmap, 0.0, 0.5)
    pcm = ax.pcolormesh(tmp_x0, tmp_x1, z,
                        norm=cm.colors.Normalize(vmin=0, vmax=
1),
                        cmap=new_cmap, shading='nearest', alpha
= 0.9)
    ax.figure.colorbar(pcm, ax=ax)
```

- Phần x0 và x1 là để vẽ decision boundary khi:
  - w0x0 + w1×1 + b = 0
  - mình tính 2 giao điểm x1=0 và x2=0
  - sau đó plot vào với một số tham số để điều chỉnh độ dày cuar đường, màu sắc.

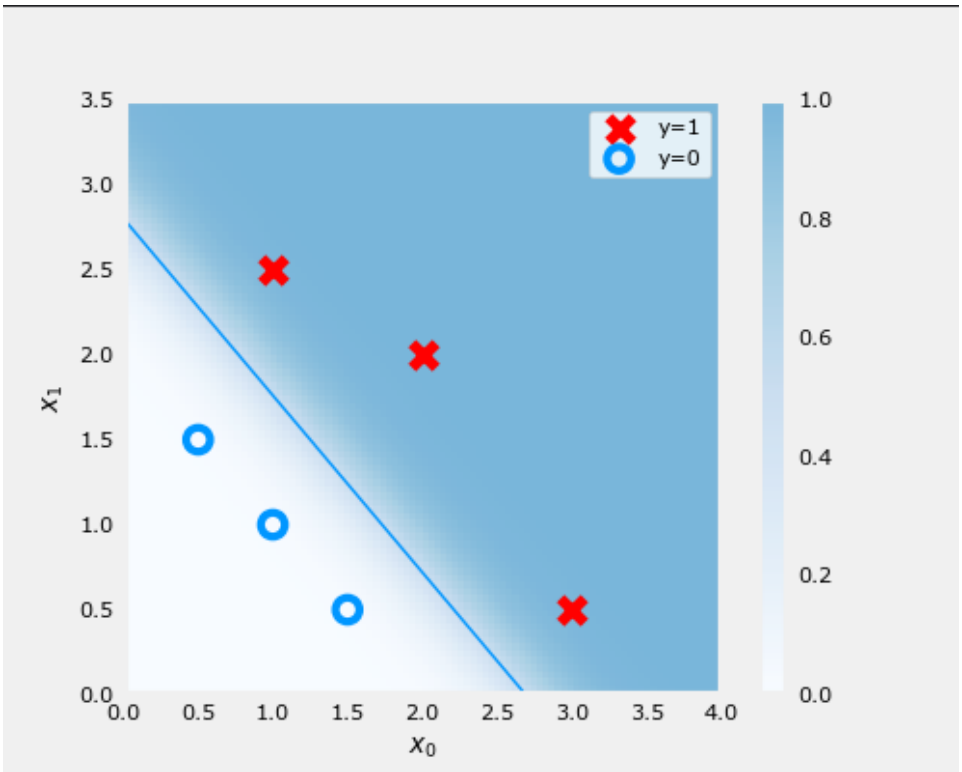
```
fig,ax = plt.subplots(1,1,figsize=(5,4))
# plot the probability
plt_prob(ax, w_out, b_out)

# Plot the original data
ax.set_ylabel(r'$x_1$')
ax.set_xlabel(r'$x_0$')
ax.axis([0, 4, 0, 3.5])
plot_data(X_train,y_train,ax)

# Plot the decision boundary
```

```
x0 = -b_out/w_out[0]
x1 = -b_out/w_out[1]
ax.plot([0,x0],[x1,0], c=dlc["dlblue"], lw=1)
plt.show()
```

Output:



- Có thể thấy với Gradient Descent đã tìm ra đúng giá trị w và b phù hợp để vẽ ra đường thẳng phân loại giữa các giá trị.